```
import os

file_path = r"G:\Prodigy InfoTech\Task 5\
US_Accidents_March23_sampled_500k.xlsx"

print(os.path.exists(file_path))
```

```
True
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import folium
from folium.plugins import HeatMap

df = pd.read_excel(
    r"G:/Prodigy InfoTech/Task
5/US_Accidents_March23_sampled_500k.xlsx",
    nrows=5000
)

print("Rows:", len(df))
print("Columns:", len(df.columns))

df.head()
```

```
Rows: 5000
Columns: 46

          ID   Source  Severity          Start_Time
End_Time  \
0  A-2047758  Source2         2 2019-06-12 10:10:56 2019-06-12
10:55:58
1  A-4694324  Source1         2 2022-12-03 23:37:14 2022-12-04
01:56:53
2  A-5006183  Source1         2 2022-08-20 13:13:00 2022-08-20
15:22:45
3  A-4237356  Source1         2 2022-02-21 17:43:04 2022-02-21
19:43:23
4  A-6690583  Source1         2 2020-12-04 01:46:00 2020-12-04
04:13:09

   Start_Lat   Start_Lng    End_Lat     End_Lng  Distance(mi)  ...
Roundabout  \
0  30.641211  -91.153481        NaN         NaN         0.000  ...
False
1  38.990562  -77.399070  38.990037  -77.398282         0.056  ...
False
2  34.661189 -120.492822  34.661189 -120.492442         0.022  ...
False
```

```
3  43.680592  -92.993317  43.680574  -92.972223         1.054  ...
False
4  35.395484 -118.985176  35.395476 -118.985995         0.046  ...
False

  Station   Stop Traffic_Calming Traffic_Signal Turning_Loop
Sunrise_Sunset  \
0   False  False           False           True        False
Day
1   False  False           False          False        False
Night
2   False  False           False           True        False
Day
3   False  False           False          False        False
Day
4   False  False           False          False        False
Night

  Civil_Twilight Nautical_Twilight Astronomical_Twilight
0            Day               Day                   Day
1          Night             Night                 Night
2            Day               Day                   Day
3            Day               Day                   Day
4          Night             Night                 Night

[5 rows x 46 columns]
```

```
print(df.columns.tolist())
```

```
['ID', 'Source', 'Severity', 'Start_Time', 'End_Time', 'Start_Lat',
'Start_Lng', 'End_Lat', 'End_Lng', 'Distance(mi)', 'Description',
'Street', 'City', 'County', 'State', 'Zipcode', 'Country', 'Timezone',
'Airport_Code', 'Weather_Timestamp', 'Temperature(F)',
'Wind_Chill(F)', 'Humidity(%)', 'Pressure(in)', 'Visibility(mi)',
'Wind_Direction', 'Wind_Speed(mph)', 'Precipitation(in)',
'Weather_Condition', 'Amenity', 'Bump', 'Crossing', 'Give_Way',
'Junction', 'No_Exit', 'Railway', 'Roundabout', 'Station', 'Stop',
'Traffic_Calming', 'Traffic_Signal', 'Turning_Loop', 'Sunrise_Sunset',
'Civil_Twilight', 'Nautical_Twilight', 'Astronomical_Twilight']
```

```
missing = df.isnull().sum().sort_values(ascending=False)
```

```
print(missing.head(20))
```

```
End_Lng              444
End_Lat              444
Precipitation(in)    303
Wind_Chill(F)        279
Wind_Speed(mph)       74
Visibility(mi)        21
Weather_Condition     20
```

```
Wind_Direction         20
Humidity(%)            17
Temperature(F)         15
Pressure(in)           14
Weather_Timestamp      13
Airport_Code            4
Street                  3
Timezone                2
County                  0
Description             0
City                    0
Start_Lng               0
Start_Time              0
dtype: int64
```

```
numerical_cols = [
    'Temperature(F)',
    'Visibility(mi)',
    'Humidity(%)',
    'Wind_Speed(mph)',
    'End_Lat',
    'End_Lng',
    'Weather_Timestamp',
    'Wind_Chill(F)',
    'Pressure(in)',
    'Precipitation(in)'
]

for col in numerical_cols:
    if col in df.columns:
        df[col] = df[col].fillna(df[col].median())

categorical_cols = [
    'Weather_Condition',
    'Wind_Direction',
    'Timezone',
    'Airport_Code',
    'Street',
    'Sunrise_Sunset',
    'Civil_Twilight',
    'Nautical_Twilight',
    'Astronomical_Twilight'
]

for col in categorical_cols:
    if col in df.columns:
        df[col] = df[col].fillna(df[col].mode()[0])

print(df.isnull().sum())
```

```
ID                        0
Source                    0
Severity                  0
Start_Time                0
End_Time                  0
Start_Lat                 0
Start_Lng                 0
End_Lat                   0
End_Lng                   0
Distance(mi)              0
Description               0
Street                    0
City                      0
County                    0
State                     0
Zipcode                   0
Country                   0
Timezone                  0
Airport_Code              0
Weather_Timestamp         0
Temperature(F)            0
Wind_Chill(F)             0
Humidity(%)               0
Pressure(in)              0
Visibility(mi)            0
Wind_Direction            0
Wind_Speed(mph)           0
Precipitation(in)         0
Weather_Condition         0
Amenity                   0
Bump                      0
Crossing                  0
Give_Way                  0
Junction                  0
No_Exit                   0
Railway                   0
Roundabout                0
Station                   0
Stop                      0
Traffic_Calming           0
Traffic_Signal            0
Turning_Loop              0
Sunrise_Sunset            0
Civil_Twilight            0
Nautical_Twilight         0
Astronomical_Twilight     0
dtype: int64
```

```
df['Start_Time'] = pd.to_datetime(df['Start_Time'])
```

```
df['Hour'] = df['Start_Time'].dt.hour

def get_period(hour):
    if 5 <= hour < 12:
        return "Morning"
    elif 12 <= hour < 17:
        return "Afternoon"
    elif 17 <= hour < 21:
        return "Evening"
    else:
        return "Night"

df['Time_of_Day'] = df['Hour'].apply(get_period)

print(df.columns.tolist())
```

```
['ID', 'Source', 'Severity', 'Start_Time', 'End_Time', 'Start_Lat',
'Start_Lng', 'End_Lat', 'End_Lng', 'Distance(mi)', 'Description',
'Street', 'City', 'County', 'State', 'Zipcode', 'Country', 'Timezone',
'Airport_Code', 'Weather_Timestamp', 'Temperature(F)',
'Wind_Chill(F)', 'Humidity(%)', 'Pressure(in)', 'Visibility(mi)',
'Wind_Direction', 'Wind_Speed(mph)', 'Precipitation(in)',
'Weather_Condition', 'Amenity', 'Bump', 'Crossing', 'Give_Way',
'Junction', 'No_Exit', 'Railway', 'Roundabout', 'Station', 'Stop',
'Traffic_Calming', 'Traffic_Signal', 'Turning_Loop', 'Sunrise_Sunset',
'Civil_Twilight', 'Nautical_Twilight', 'Astronomical_Twilight',
'Hour', 'Time_of_Day']
```

```
df['Start_Time'] = pd.to_datetime(df['Start_Time'])

df['Hour'] = df['Start_Time'].dt.hour

print(df[['Start_Time', 'Hour']].head())
```

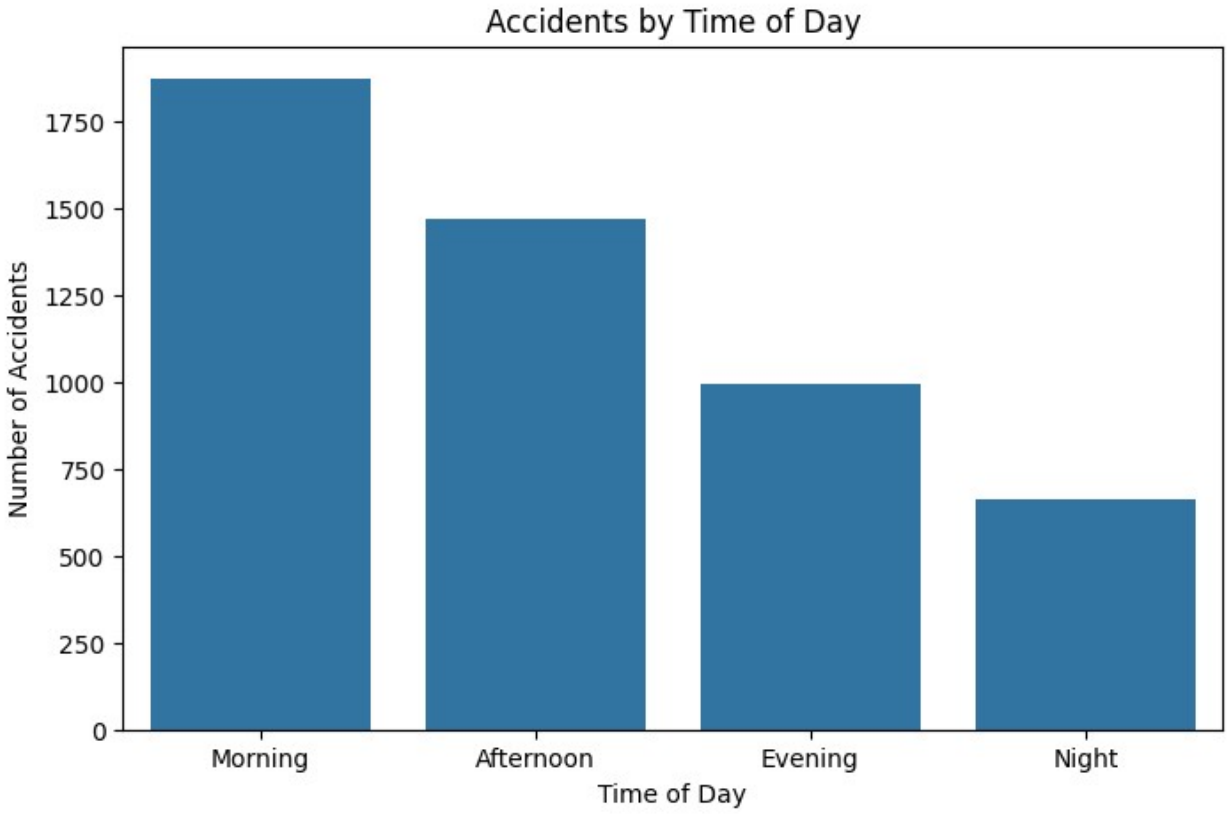
```
           Start_Time  Hour
0 2019-06-12 10:10:56    10
1 2022-12-03 23:37:14    23
2 2022-08-20 13:13:00    13
3 2022-02-21 17:43:04    17
4 2020-12-04 01:46:00     1
```

```
def get_period(hour):
    if 5 <= hour < 12:
        return 'Morning'
    elif 12 <= hour < 17:
        return 'Afternoon'
    elif 17 <= hour < 21:
        return 'Evening'
    else:
        return 'Night'
```

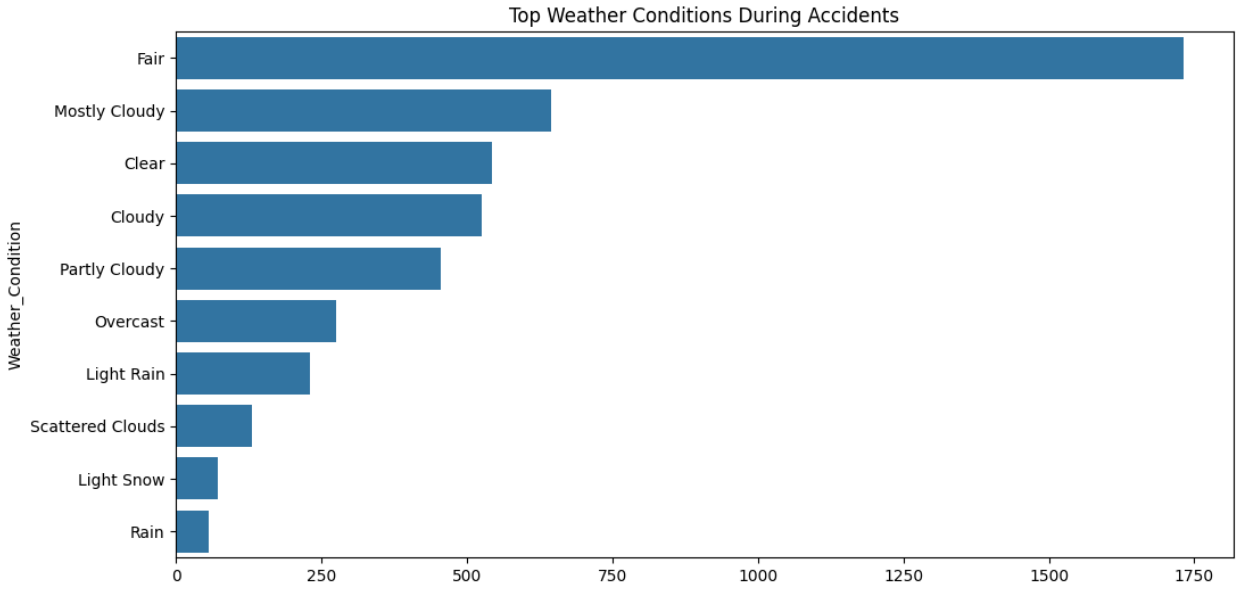
```
df['Time_of_Day'] = df['Hour'].apply(get_period)

print(df['Time_of_Day'].value_counts())
```

```
Time_of_Day
Morning      1874
Afternoon    1470
Evening       993
Night         663
Name: count, dtype: int64
```

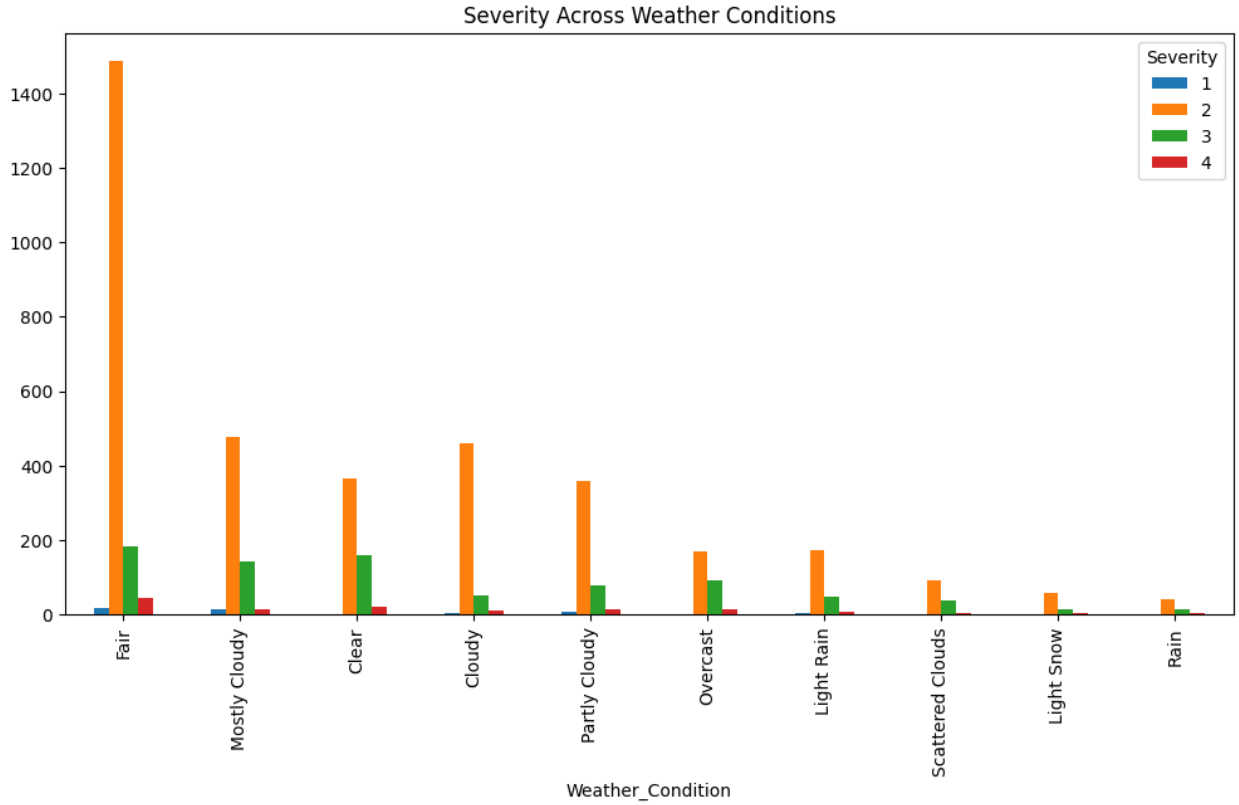
```
plt.figure(figsize=(8,5))

sns.countplot(
    data=df,
    x='Time_of_Day',
    order=['Morning','Afternoon','Evening','Night']
)

plt.title('Accidents by Time of Day')
plt.xlabel('Time of Day')
plt.ylabel('Number of Accidents')

plt.show()
```



```
top_weather = df['Weather_Condition'].value_counts().head(10)

plt.figure(figsize=(12,6))

sns.barplot(
    x=top_weather.values,
    y=top_weather.index
)

plt.title("Top Weather Conditions During Accidents")

plt.show()
```

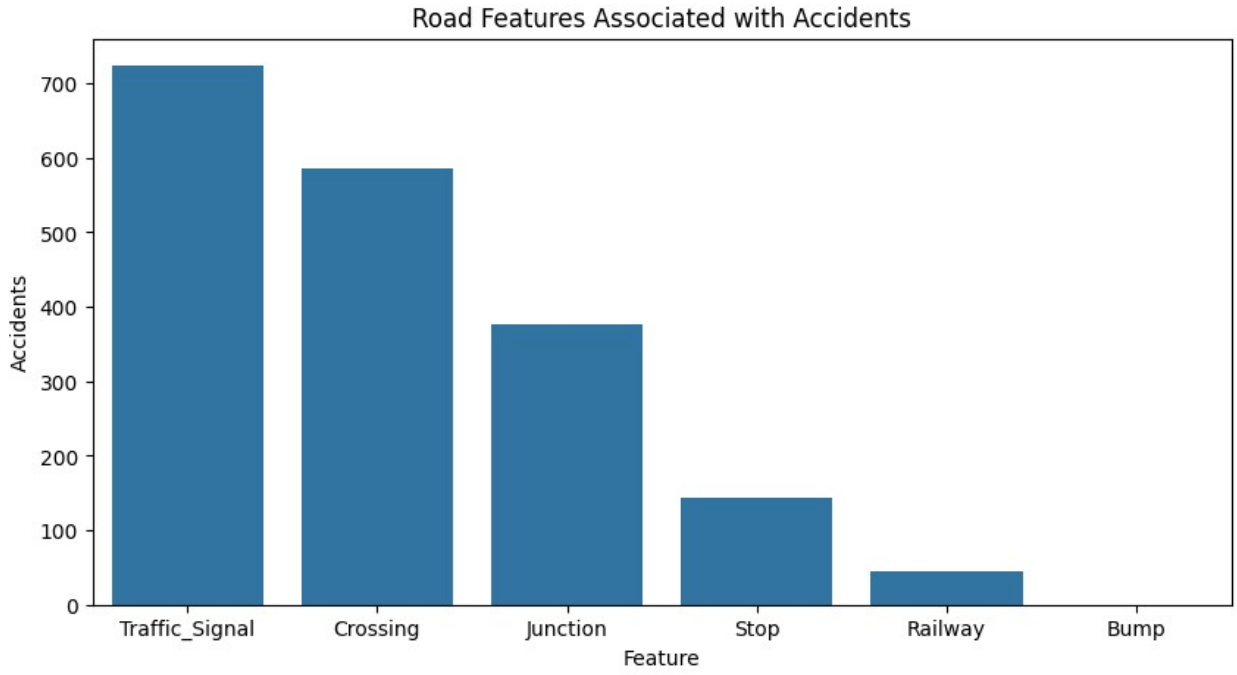



```
weather_severity = pd.crosstab(
    df['Weather_Condition'],
    df['Severity']
)

top_weather = df['Weather_Condition'].value_counts().head(10).index

weather_severity.loc[top_weather].plot(
    kind='bar',
    figsize=(12,6)
)

plt.title("Severity Across Weather Conditions")

plt.show()
```

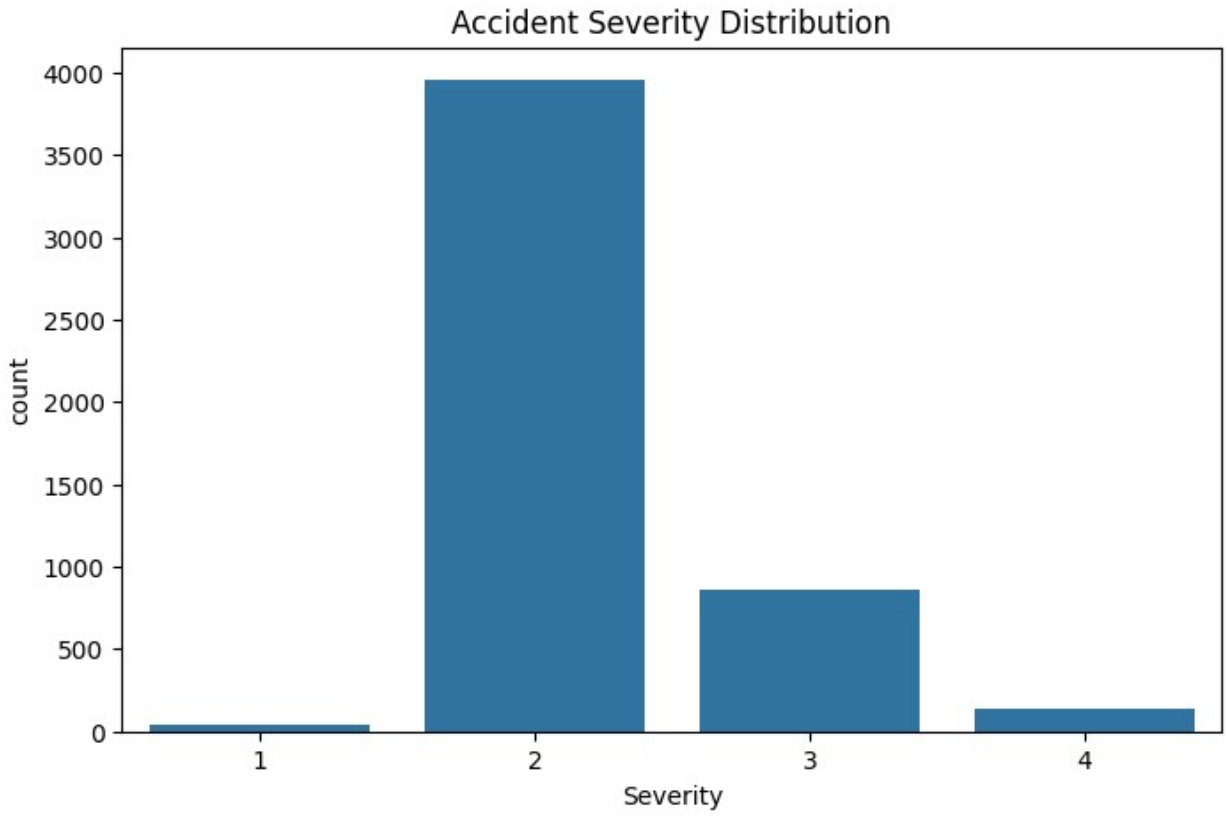



```
road_features = [
    'Junction',
    'Traffic_Signal',
    'Stop',
    'Crossing',
    'Bump',
    'Railway'
]

road_counts = {}

for col in road_features:
    if col in df.columns:
        road_counts[col] = df[col].sum()

road_df = pd.DataFrame(
    road_counts.items(),
    columns=['Feature','Accidents']
)

road_df = road_df.sort_values(
    by='Accidents',
    ascending=False
)
```

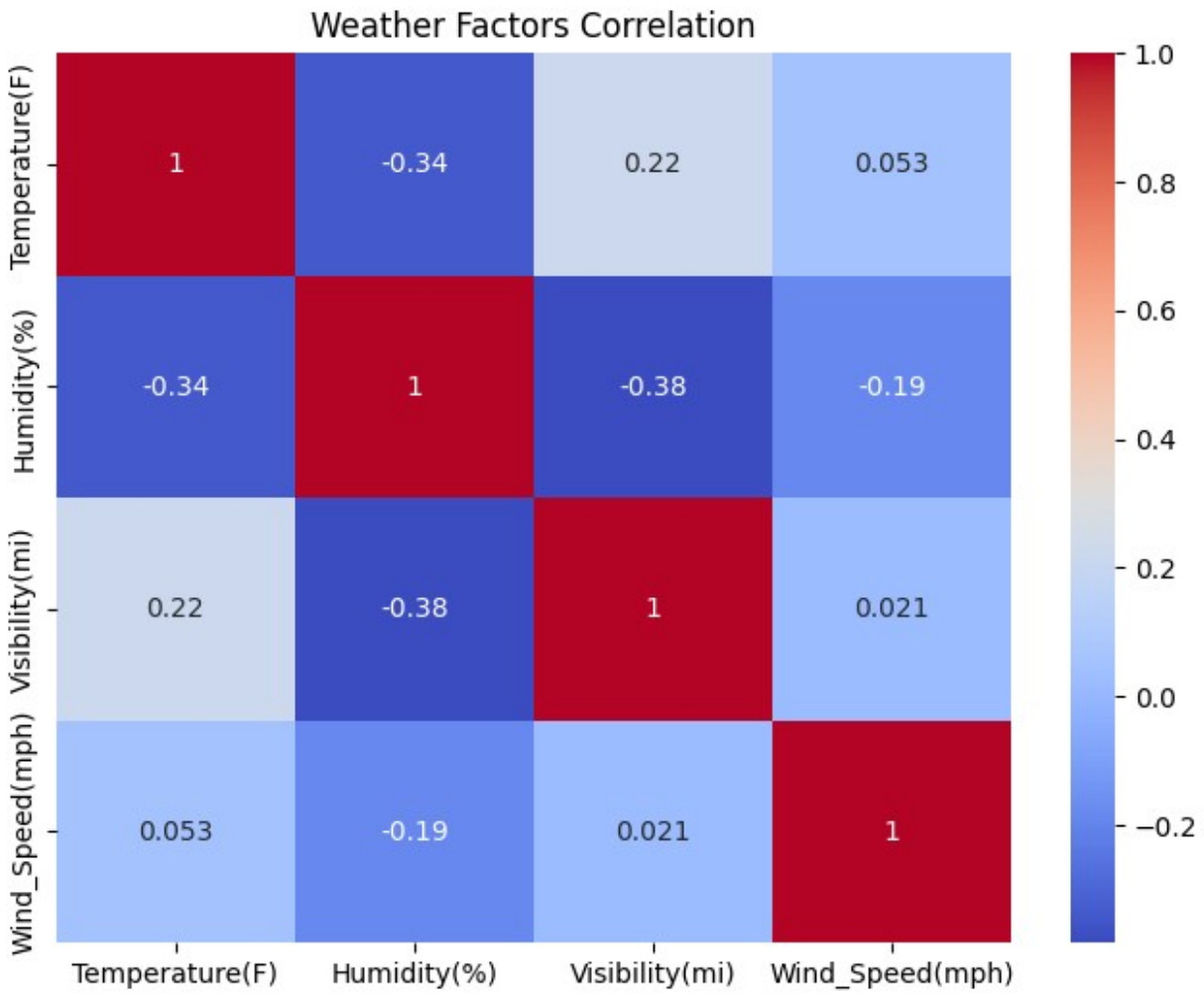
```
plt.figure(figsize=(10,5))

sns.barplot(
    data=road_df,
    x='Feature',
    y='Accidents'
)

plt.title("Road Features Associated with Accidents")

plt.show()
```




```
plt.figure(figsize=(8,5))

sns.countplot(
    data=df,
    x='Severity'
)

plt.title("Accident Severity Distribution")

plt.show()
```




```
weather_cols = [
    'Temperature(F)',
    'Humidity(%)',
    'Visibility(mi)',
    'Wind_Speed(mph)'
]

corr = df[weather_cols].corr()

plt.figure(figsize=(8,6))

sns.heatmap(
    corr,
    annot=True,
    cmap='coolwarm'
)

plt.title("Weather Factors Correlation")

plt.show()
```

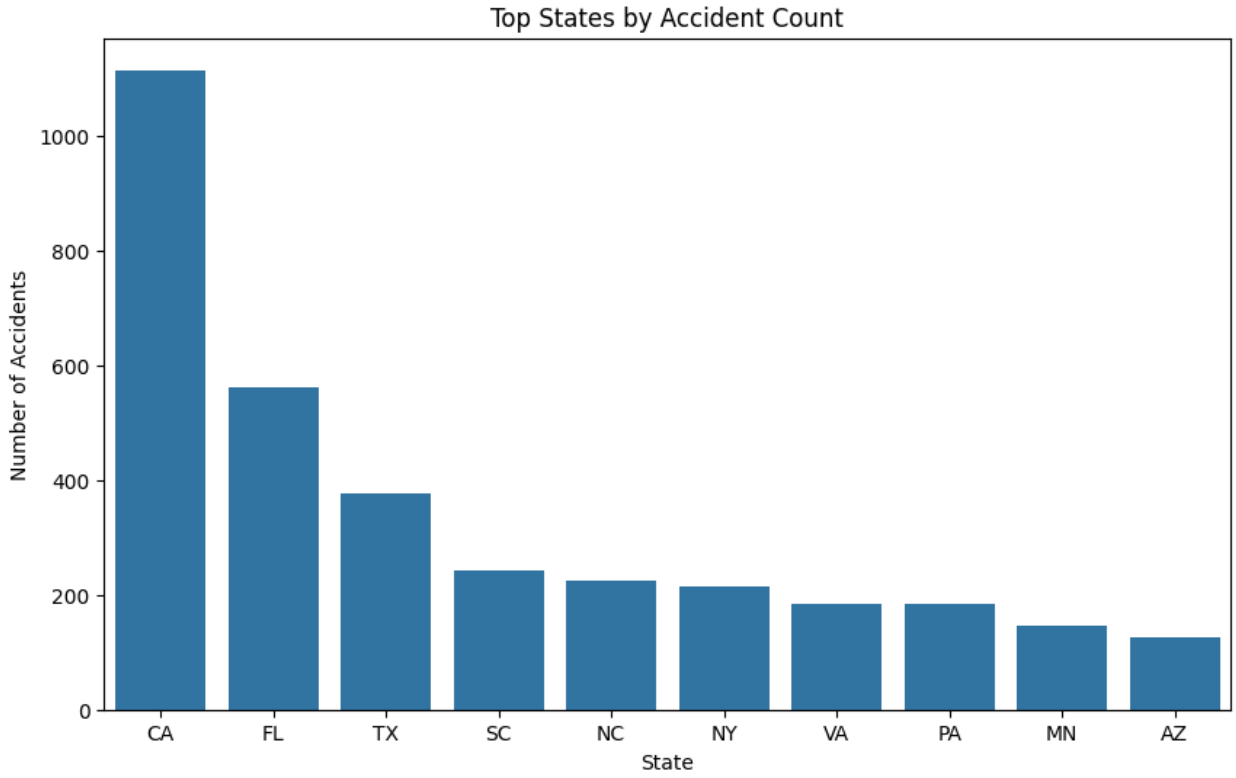



```
print(df[['Start_Lat', 'Start_Lng']].head())

sample_df = df[['Start_Lat', 'Start_Lng']].dropna()
sample_df = sample_df.sample(
    min(5000, len(sample_df)),
    random_state=42
)

m = folium.Map(
    location=[
        sample_df['Start_Lat'].mean(),
        sample_df['Start_Lng'].mean()
    ],
    zoom_start=5
)

HeatMap(sample_df.values.tolist()).add_to(m)

m
```

```
   Start_Lat   Start_Lng
0  30.641211  -91.153481
1  38.990562  -77.399070
2  34.661189 -120.492822
3  43.680592  -92.993317
4  35.395484 -118.985176
```

```
<folium.folium.Map at 0x1e2e7c01090>
```

```
top_states = df['State'].value_counts().head(10)

plt.figure(figsize=(10,6))

sns.barplot(
    x=top_states.index,
    y=top_states.values
)

plt.title("Top States by Accident Count")

plt.ylabel("Number of Accidents")

plt.show()
```




```
severity_time = pd.crosstab(
    df['Time_of_Day'],
```

```
    df['Severity']
)

severity_time.plot(
    kind='bar',
    figsize=(10,6)
)

plt.title("Severity by Time of Day")

plt.show()
```